🎬

Case Study — THE CONFRONTATION CLIP on a Free Kaggle T4 (WAN 2.2 I2V)

🎯 A before/after engineering case study of rendering a 26-shot cinematic clip on a **free** dual-T4 Kaggle notebook using WAN 2.2 I2V. The headline: the GPU was never the bottleneck — a single misconfigured postprocess line was, and fixing it cut the projected render from ~33 h to ~5–6 h at **$0 cost**.

1. Initial observations from the output folders (creator's field notes)

These are the firsthand observations from inspecting the actual output folders of that run — the raw signal that seeded the whole analysis below.

👁️ **Observation 1 — the raw beats the final.** `take_0` (the raw WAN output) is visibly **sharper and higher quality** than the delivered `final.mp4`. The postprocess chain is *subtracting* quality, not adding it.
Observation 2 — no real motion. There is **no actual motion beyond the original gesture already posed in the source image**. The clip animates the pose it was handed and little else — the shot is effectively a living still.
Observation 3 — costume held. Across the whole clip the character kept the **same costume** — identity/wardrobe consistency was preserved end-to-end, confirming the stills anchor is doing its job on the *consistency* axis even while motion stays flat.

The creator's architecture & proposition (in their own words)

The observations above converged into an explicit design thesis, stated firsthand during the session. Reproduced verbatim, then unpacked:

> 💬 *"Why not allow WAN model to exercise full freedom — rest of quality and other hacks we do offline using well-established image and video processing algorithms."*
>
> *"Consistency can be interpolated across frames. Let WAN do the heavy lifting — the actual generation... I think going by the initial stills generation [is] the key professional cinema approach."*

Unpacked, this is a **keyframe-driven generative-film architecture** — and it maps 1:1 onto the professional order of operations (storyboard → key art → animate → finish):

1. **The initial stills are the key.** The frame is the unit of craft: composition, identity, costume, and lighting are all baked into the anchor still. It's also where quality-per-compute is highest — fixing one frame is cheap and deterministic; fixing 41 moving frames is not. Front-load quality into the still and the whole shot inherits it. *(This is the strongest insight — it's ~80% of the battle.)*
2. **Let WAN exercise full freedom on motion.** Anchor on the still, then let the model *generatively* interpolate the in-betweens instead of clamping it to a living photo. WAN owns the temporal axis; the still only sets the spatial ceiling.
3. **Do the rest of the quality work offline, with established algorithms.** Grade, denoise, upscale as a *finishing* pass after download — off the GPU quota, using well-understood image/video tooling. Finishing, never rescue.
4. **"Consistency can be interpolated" — but only the right kind.** *Generative* interpolation (WAN creating semantically coherent in-between frames) is the real thing and builds consistency *within* a shot. *Optical* interpolation (RIFE / ffmpeg `minterpolate`) does **not** create consistency — it blends pixels and amplifies artifacts, which is exactly what degraded `final.mp4`. Across hard cuts nothing interpolates at all — which is *why* consistent stills are the foundation.

✅ **"Is this architecture already implemented?" — Yes, verified in the kit source.** The default render mode *is* the blueprint path (`KIT_RENDER_MODE=motion_stills`: "stills first, then animate each still into motion"); stills are a dedicated first-class stage; cross-shot `flf2v` chaining exists; identities are forged up front. **The creator independently reverse-engineered the exact architecture the kit already ships.** The catch: it's the default *tuning* that undercut it — motion `denoise` defaulted to `0.35`, end-frame morph was OFF by default (hence the frozen poses), the stills stage ran at economy settings, and the lossy CPU post (incl. `minterpolate`) was on. So the fix is **parameter tuning, not a rebuild** — a config problem sitting on top of a sound design.

2. What was rendered

- **Film:** `THE_CONFRONTATION_CLIP` — 26 shots across 3 scored movements.
  - **Prelude** (4): pro_alone, pro_seen, pro_silence, pro_crossing
  - **Search** (5): search_void, search_run, search_water, search_fear, search_found
  - **Confrontation** (17): confr_caught → confr_fall
- **Engine:** WAN 2.2 I2V A14B GGUF (primary), LTXV fallback only. Native output **16 fps**.
- **Hardware:** free Kaggle session — 2× Tesla T4 (16 GB each), sharded one ComfyUI server per GPU.
- **Cast:** jack + julie, identity via SDXL character-forge (no source photos).

3. Where the time actually went (the core finding)

Stage	What it does	Verdict
GPU generation	WAN diffusion inference, ~48 passes over the shot list	✅ Healthy — ~4.56 h GPU, ~5.7 min/pass, zero errors

Stage	What it does	Verdict
CPU postprocess	`interpolate_fps: 60` forced ffmpeg motion-interpolation 16→60 fps **per shot**	❌ The killer — the real bottleneck

Measured fix: the per-clip postprocess dropped from **4 m 36 s → 44 s** on a 2-second clip (~6× faster) once interpolation was matched to WAN's native frame rate.

> ⚡ **Net effect:** projected full-film time fell from **~33 h → ~5–6 h** with *no* change to the GPU workload or output quality. The lesson: profile the whole pipeline, not just the GPU — the accelerator was innocent the entire time.

4. Cost analysis

Plan	Card	Takes	Est. time	Cost
Full film, full quality	2× T4 (Kaggle)	3	~8–11 h wall / 2–3 free weekly sessions	**$0**
Full film, cost-cut	1× L4 (Lightning)	1	~6–10 GPU-hr	~$3–5

The pipeline gates spend with a **cost meter**: it renders one sample shot, prints measured per-shot time + projected total, then HOLDs until `CONFIRM_FULL_RUN=1`. You can never blind-burn credits. On Kaggle the whole thing is $0.

5. Config vs. runtime — the gap

A recurring theme: the static `config_confrontation.yaml` and what actually ran diverged. This gap caused real confusion during debugging and is worth naming explicitly.

Setting	Config file	Actually ran	Note
`active_profile`	`lightning_l4`	`kaggle_2xt4`	Notebook `sed`-patches it at launch

Setting	Config file	Actually ran	Note
`interpolate_fps`	was `60` (now fixed to `16`)	`60` during the slow run	The bottleneck; now matched to WAN's 16 fps
`upscale`	`2.0`	forced to `1.0`	Notebook lets the graph/HD-pass handle it
`steps`	`40`	`28` (`KIT_MOTION_STEPS`)	Runtime override
Resolution	`1280×720`	`768×512` (raw take)	Inline graph dimensions ≠ config

Takeaway: the notebook is the source of truth at runtime, not the YAML. Any future tuning must be verified against the notebook's overrides, or the config change is silently ignored.

6. Quality tradeoffs

- `take_0` **(raw) looked *sharper* than** `final` **(postprocessed).** The CPU upscale + grade pass was *softening* detail rather than adding it. → Skip per-shot spatial upscale; do a single HD upscale on CPU after download (zero GPU quota).
- **Motion was "stuck to pose."** WAN I2V anchored too tightly to the forged start frame → static shots. → Lower conditioning fps and/or raise `denoise` so WAN does more of the motion "heavy lifting."
- **Interpolate once, not per shot.** Deliver at native 16 fps per shot; do any smooth-motion interpolation once at final assembly, not 26 times mid-pipeline.

7. Tuning recommendations for the next dual-T4 run

Lever	Current	Recommended	Why
`interpolate_fps`	16 ✅	keep 16	Skips the CPU minterpolate that killed the run
Per-shot `upscale`	1.0 (runtime)	keep off	`take_0` proved raw is sharper; upscale on CPU later
`denoise`	0.55	try 0.60–0.65 on static shots	More motion, less pose-lock

Lever	Current	Recommended	Why
`steps`	28 (runtime)	28–32	Diminishing returns above ~32 on T4
Final interpolation	per shot	once at stitch	Smoothness without per-shot CPU tax

8. The bigger picture

> The film without the architecture story is forgettable. The architecture without the film is unconvincing. Together they're a case study.

What this project actually demonstrates: a full multi-character, scored, voiced cinematic pipeline running end-to-end on **free** consumer-tier cloud GPUs, with identity invented from prompts (no photos), resumable across sessions, and gated so it can never overspend. The "means of production" for short AI film are now effectively free to anyone who shows up with the engineering discipline to tune the pipeline — which is exactly what the before/after above proves.

📌 **Data provenance:** Config, profile, and shot numbers were read live from the kit source this session. Generation/postprocess timings (4.56 h GPU, 4 m 36 s → 44 s) come from the previously analyzed T4 run log — the raw logs were not re-uploaded this session, so those figures are from prior verified analysis, not freshly re-measured. Re-run the cost meter on the next session to confirm against current hardware.